



Database systems

ITCS2322

Chapter 12 : Distributed Database Management Systems

اعداد : دكتور محمد عبد اللطيف راضي

Learning Objectives

- In this chapter, the student will learn:
 - About distributed database management systems (DDBMSs) and their components
 - How database implementation is affected by different levels of data and process distribution
 - How transactions are managed in a distributed database environment

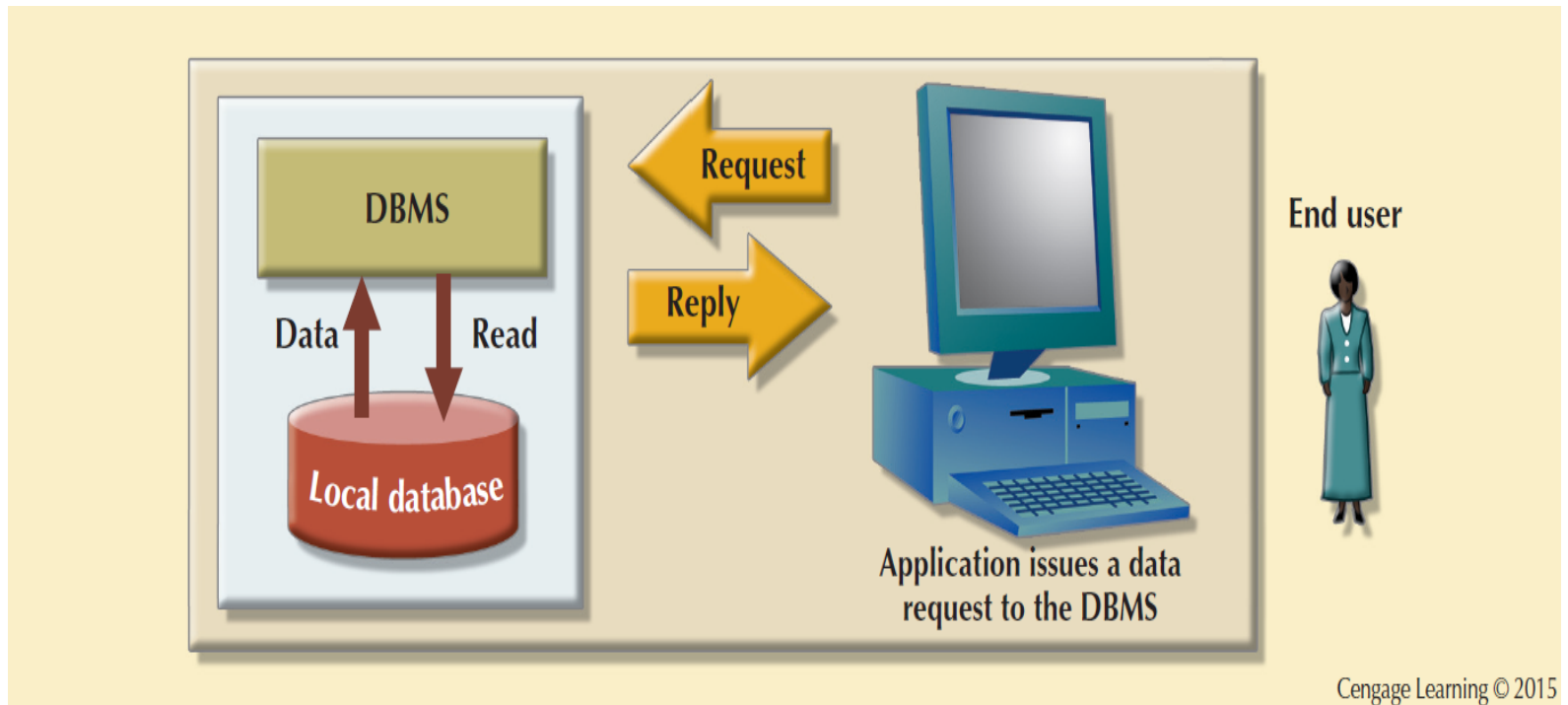
Learning Objectives

- In this chapter, the student will learn:
 - How distributed database design draws on data partitioning and replication to balance performance, scalability, and availability
 - About the trade-offs of implementing a distributed data system

Evolution Database Management Systems

- Distributed database management system (DDBMS): Governs storage and processing of logically related data over interconnected computer systems
 - Data and processing functions are distributed among several sites
- Centralized database management system
 - Required that corporate data be stored in a single central site

Figure 12.1 - Centralized Database Management System



Factors Affecting the Centralized Database Systems

- Globalization of business operation
- Advancement of web-based services
- Rapid growth of social and network technologies
- Digitization resulting in multiple types of data
- Innovative business intelligence through analysis of data

Factors Affecting the Centralized Database Systems

- Two database requirements became obvious:
 - Rapid ad hoc data access became crucial in the quick-response decision-making environment.
 - Distributed data access was needed to support geographically dispersed business units.

Factors That Aided DDBMS to Cope With Technological Advancement

Acceptance of Internet as a platform for business

Mobile wireless revolution

Usage of application as a service

Focus on mobile business intelligence

Desirability of Distributed DBMS Over Centralized DBMS

Performance
degradation

High costs

Reliability
problems

Scalability
problems

Organizational
rigidity

Advantages and Disadvantages of DDBMS

Advantages

- Data are located near greatest demand site
- Faster data access and processing
- Growth facilitation
- Improved communications
- Reduced operating costs
- User-friendly interface
- Less danger of a single-point failure
- Processor independence

Disadvantages

- Complexity of management and control
- Technological difficulty
- Security
- Lack of standards
- Increased storage and infrastructure requirements
- Increased training cost
- Costs incurred due to the requirement of duplicated infrastructure

Distributed Processing and Distributed Databases

- Distributed processing: Database's logical processing is shared among two or more physically independent sites via network
- Distributed database: Stores logically related database over two or more physically independent sites via computer network
- Database fragments: Database composed of many parts in distributed database system

Characteristics of Distributed Management Systems

A DBMS must have at least the following functions to be classified as distributed:

- **Application interface** to interact with the end user, application programs, and other DBMSs within the distributed database
- **Validation to analyse** data requests for syntax correctness
- **Transformation** to decompose complex requests into atomic data request components
- **Query optimization** to find the best access strategy (which database fragments must be accessed by the query, and how must data updates, if any, be synchronized?)
- **Mapping** to determine the data location of local and remote fragments

Characteristics of Distributed Management Systems

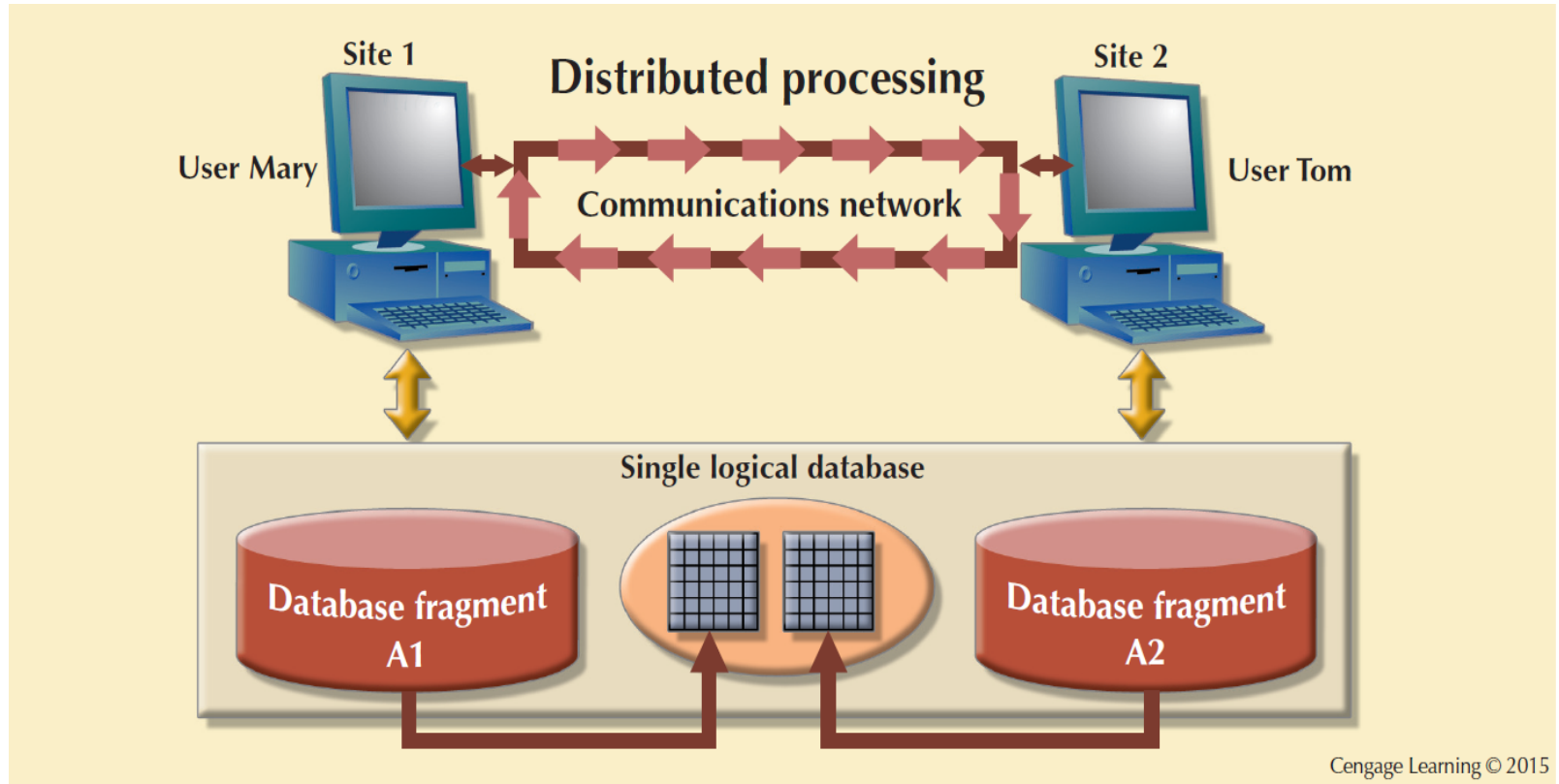
A DBMS must have at least the following functions to be classified as distributed:

- **I/O interface** to read or write data from or to permanent local storage
- **Formatting** to prepare the data for presentation to the end user or to an application program
- **Security** to provide data privacy at both local and remote databases
- **Backup and recovery** to ensure the availability and recoverability of the database in case of a failure
- **DB administration** features for the database administrator
- **Concurrency control** to manage simultaneous data access and to ensure data consistency across database fragments in the DDBMS.
- **Transaction management** to ensure that the data moves from one consistent state to another; this activity includes the synchronization of local and remote transactions as well as transactions across multiple distributed segments

Functions of Distributed DBMS

- Receives the request of an application
- **Validates analyzes**, and decomposes the request
- Maps the request
- Decomposes request into several I/O operations
- Searches and validates data
- Ensures consistency, security, and integrity
- Validates data for specific conditions
- Presents data in required format

Figure 12.4 - A Fully Distributed Database Management System



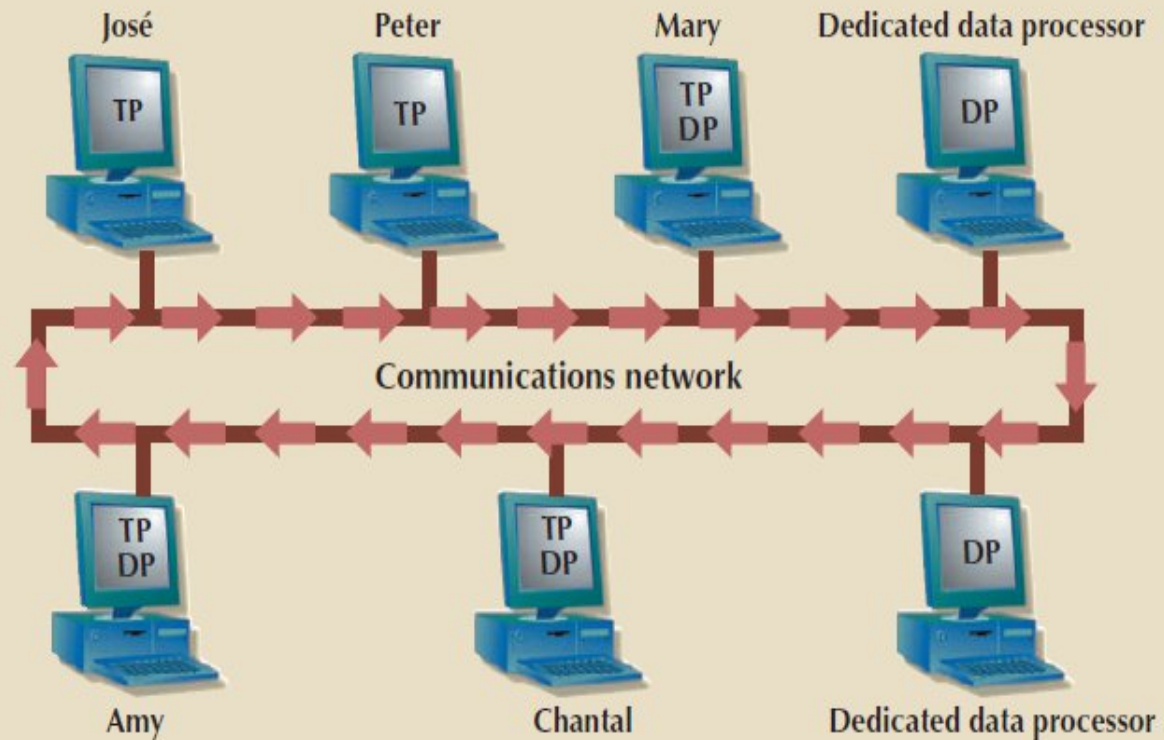
Cengage Learning © 2015

DDBMS Components

- Computer workstations or remote devices
- Network hardware and software components
- Communications media
- Transaction processor (TP): Software component of a system that requests data
- Known as transaction manager (TM) or application processor (AP)
- Data processor (DP) or data manager (DM)
- Software component on a system that stores and retrieves data from its location

DDBMS Components

FIGURE 12.5 DISTRIBUTED DATABASE SYSTEM COMPONENTS



Note: Each TP can access data on any DP, and each DP handles all requests for local data from any TP.

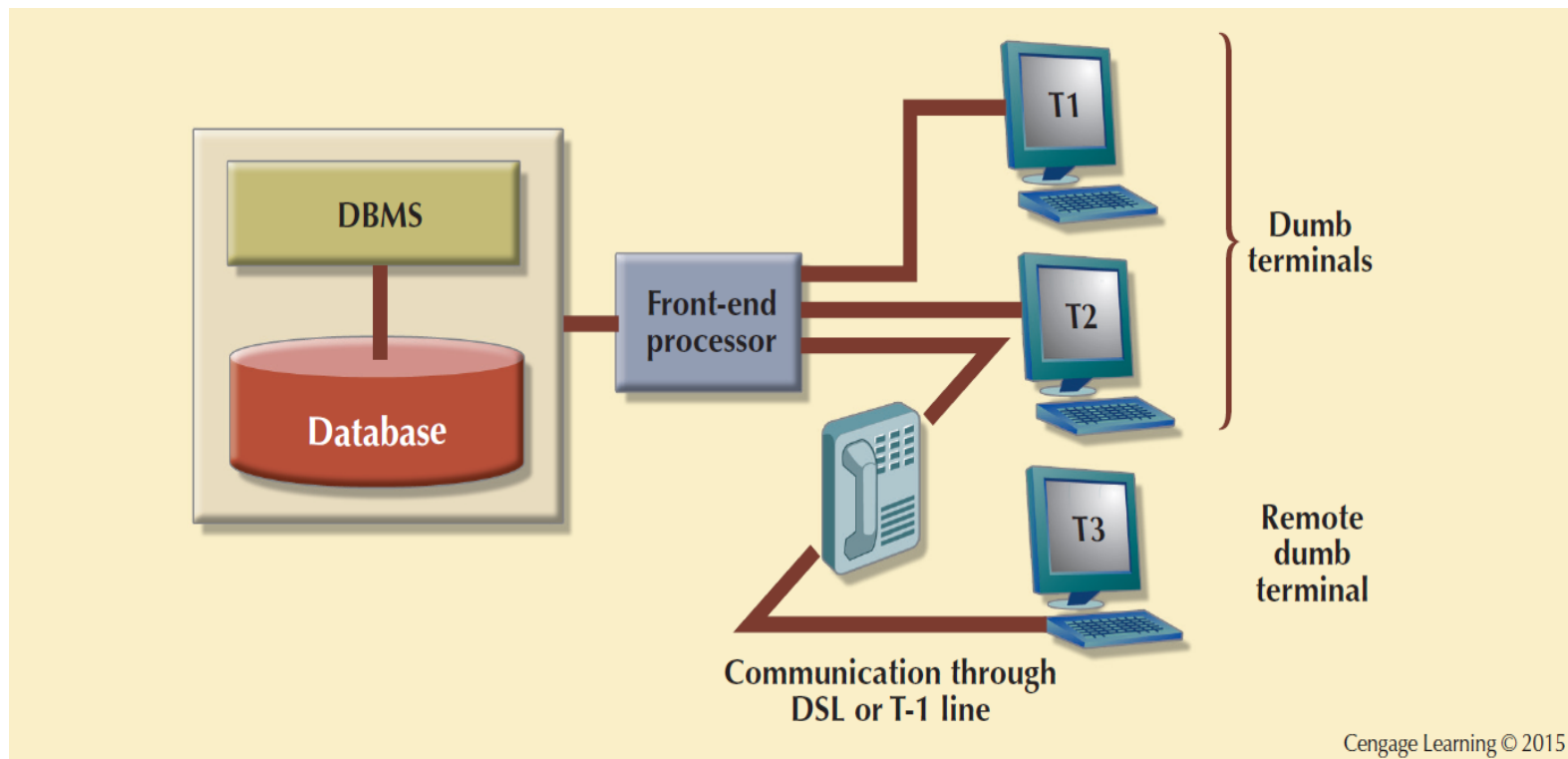
Levels of Data and Process Distribution

- Current database systems can be classified on the basis of how process distribution and data distribution are supported.
 - Single-Site Processing, Single-Site Data
 - Multiple-Site Processing, Single-Site Data
 - Multiple-Site Processing, Multiple-Site Data

Single-Site Processing, Single-Site Data (SPSD)

- Processing is done on a single host computer
- Data stored on host computer's local disk
- Processing restricted on end user's side
- DBMS is accessed by dumb terminals

Figure 12.6 - Single-Site Processing, Single-Site Data (Centralized)

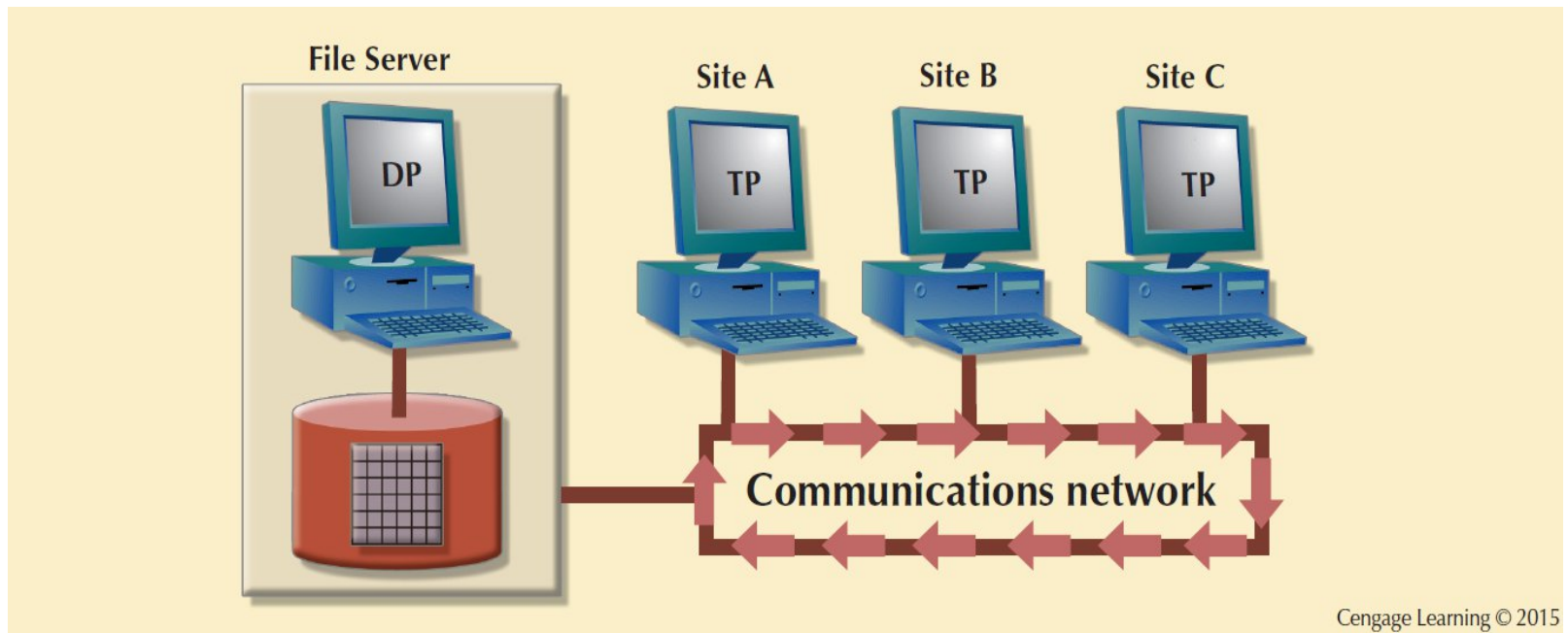


Cengage Learning © 2015

Multiple-Site Processing, Single-Site Data (MPSD)

- Multiple processes run on different computers sharing a single data repository
- Require network file server running conventional applications
 - Accessed through LAN
- Client/server architecture
 - Reduces network traffic
 - Processing is distributed
 - Supports data at multiple sites

Figure 12.7 - Multiple-Site Processing, Single-Site Data



Multiple-Site Processing, Multiple-Site Data (MPMD)

- Fully distributed database management system
 - Support multiple data processors and transaction processors at multiple sites
- Classification of DDBMS depending on the level of support for various types of databases
 - Homogeneous: Integrate multiple instances of same DBMS over a network
 - Heterogeneous: Integrate different types of DBMSs
 - Fully heterogeneous: Support different DBMSs, each supporting different data model

Restrictions of DDBMS

- Remote access is provided on a read-only basis
- Restrictions on the number of remote tables that may be accessed in a single transaction
- Restrictions on the number of distinct databases that may be accessed
- Restrictions on the database model that may be accessed

Distributed Database Transparency Features

Distribution
transparency

Transaction
transparency

Failure
transparency

Performance
transparency

Heterogeneity
transparency

Distribution Transparency

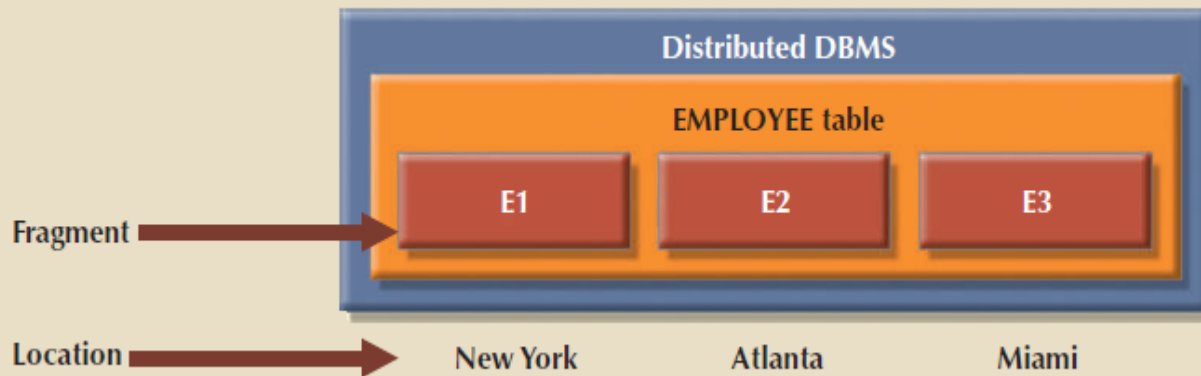
- Allows management of physically dispersed database as if centralized
- Levels
 - Fragmentation transparency
 - Location transparency
 - Local mapping transparency

Distribution Transparency

- Unique fragment: Each row is unique, regardless of the fragment in which it is located
- Supported by distributed data dictionary (DDD) or distributed data catalog (DDC)
 - DDC contains the description of the entire database as seen by the database administrator
- Distributed global schema: Common database schema to translate user requests into subqueries

Distribution Transparency

FIGURE 12.8 FRAGMENT LOCATIONS



Case 1: The Database Supports Fragmentation Transparency

The query conforms to a nondistributed database query format; that is, it does not specify fragment names or locations. The query reads:

```
SELECT      *
FROM        EMPLOYEE
WHERE       EMP_DOB < '01-JAN-1979';
```

Distribution Transparency

Case 2: The Database Supports Location Transparency

Fragment names must be specified in the query, but the fragment's location is not specified. The query reads:

```
SELECT      *
FROM        E1

WHERE       EMP_DOB < '01-JAN-1979'
UNION
SELECT      *
FROM        E2
WHERE       EMP_DOB < '01-JAN-1979'
UNION
SELECT      *
FROM        E3
WHERE       EMP_DOB < '01-JAN-1979'
```

Distribution Transparency

Case 3: The Database Supports Local Mapping Transparency

Both the fragment name and its location must be specified in the query. Using pseudo-SQL:

```
SELECT      *  
FROM        E1 NODE NY  
WHERE       EMP_DOB < '01-JAN-1979';  
UNION  
SELECT      *  
FROM        E2 NODE ATL  
WHERE       EMP_DOB < '01-JAN-1979';  
UNION  
SELECT      *  
FROM        E3 NODE MIA  
WHERE       EMP_DOB < '01-JAN-1979';
```

Distribution Transparency

TABLE 12.4

SUMMARY OF TRANSPARENCY FEATURES

IF THE SQL STATEMENT REQUIRES:

FRAGMENT NAME?	LOCATION NAME?	THEN THE DBMS SUPPORTS	LEVEL OF DISTRIBUTION TRANSPARENCY
Yes	Yes	Local mapping transparency	Low
Yes	No	Location transparency	Medium
No	No	Fragmentation transparency	High

Transaction Transparency

- Ensures database transactions will maintain distributed database's integrity and consistency
- Ensures transaction completed only when all database sites involved complete their part
- Distributed database systems require complex mechanisms to manage transactions

Distributed Requests and Distributed Transactions

Remote request

- Single SQL statement accesses data processed by a single remote database processor

Remote transaction

- Accesses data at single remote site composed of several requests

Distributed transaction

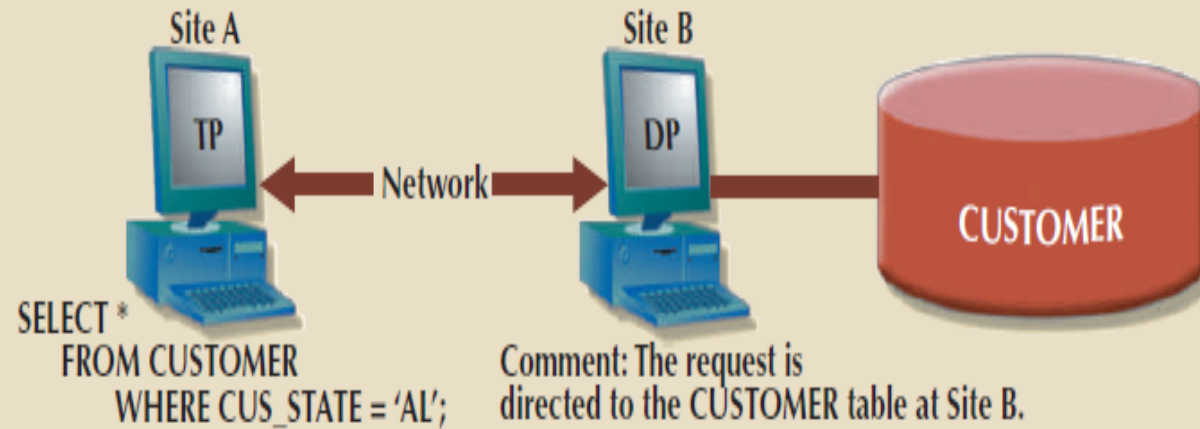
- Requests data from several different remote sites on network

Distributed request

- Single SQL statement references data at several DP sites

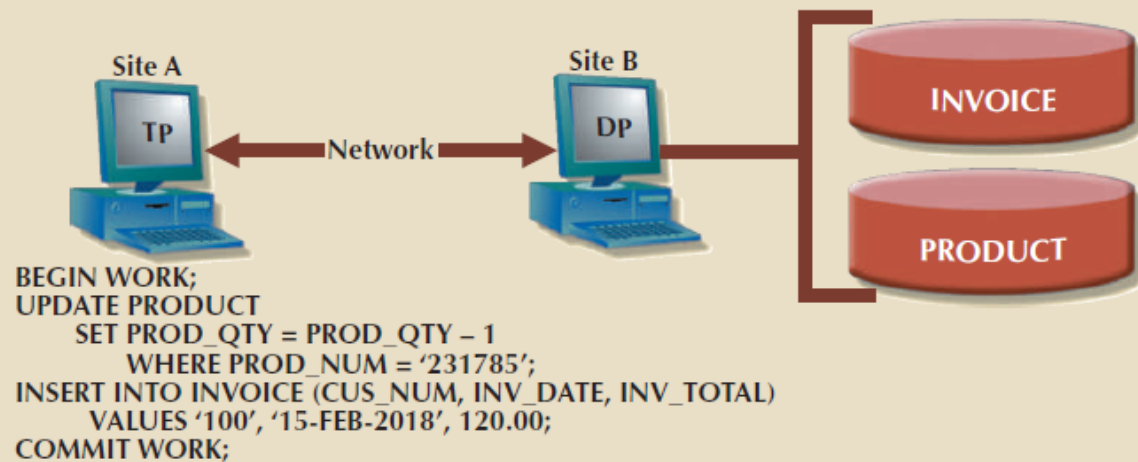
A remote request

FIGURE 12.9 A REMOTE REQUEST



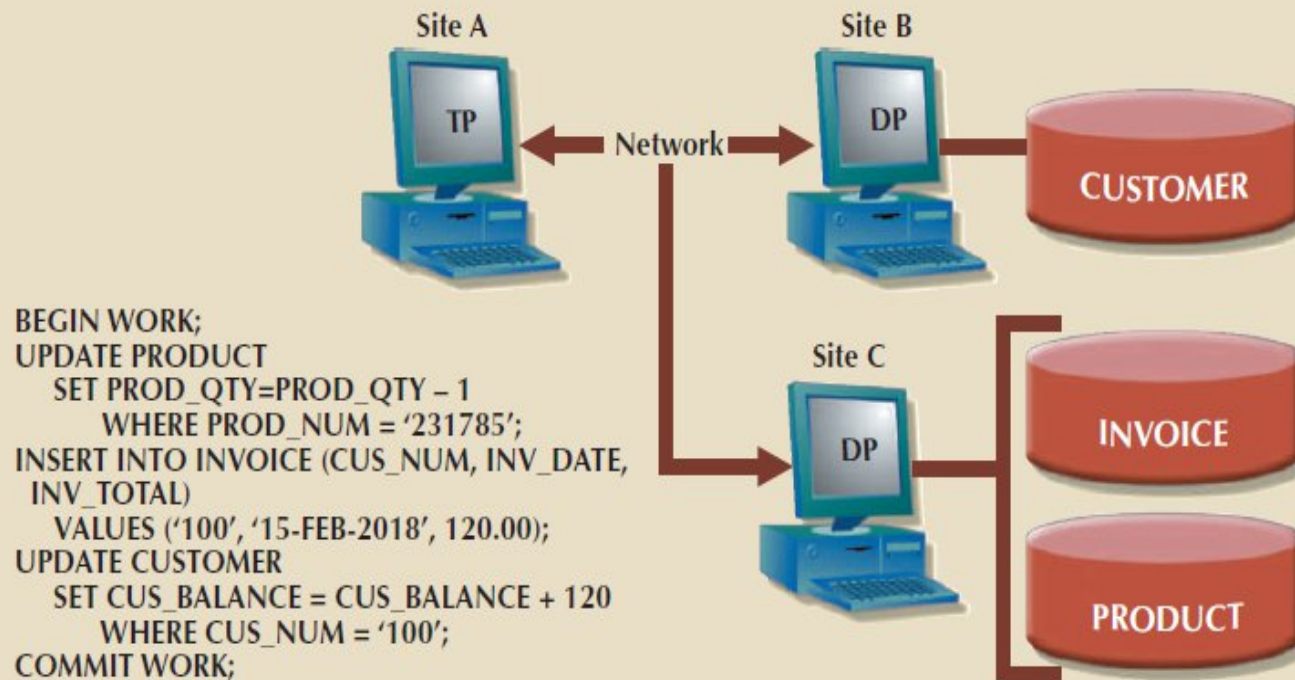
A remote transaction

FIGURE 12.10 A REMOTE TRANSACTION



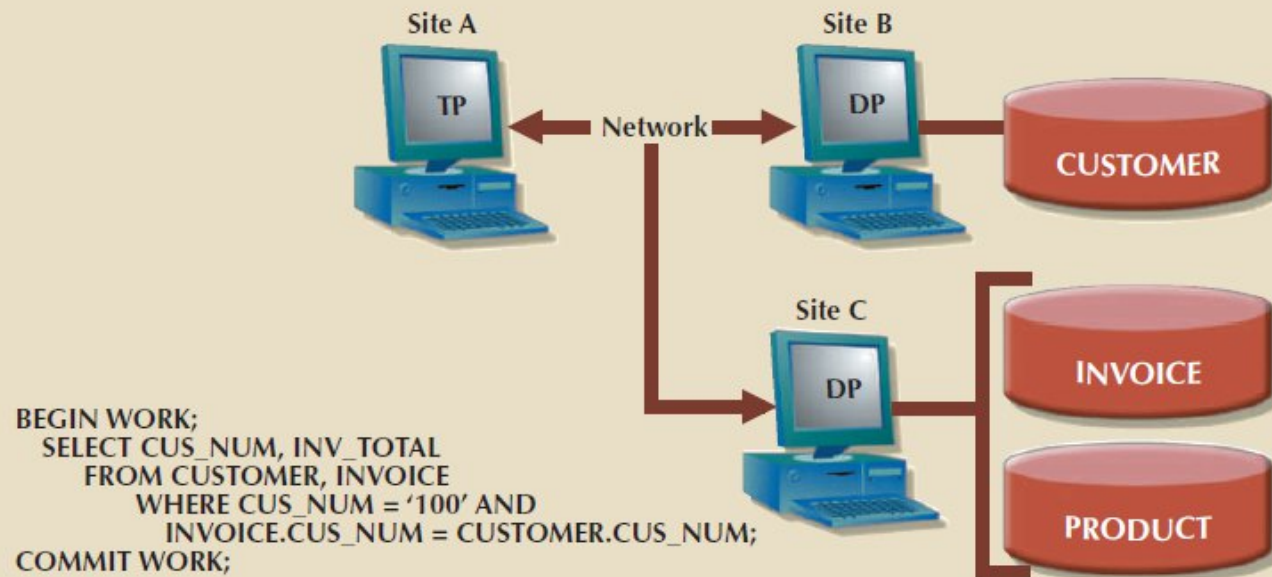
A distributed transaction

FIGURE 12.11 A DISTRIBUTED TRANSACTION



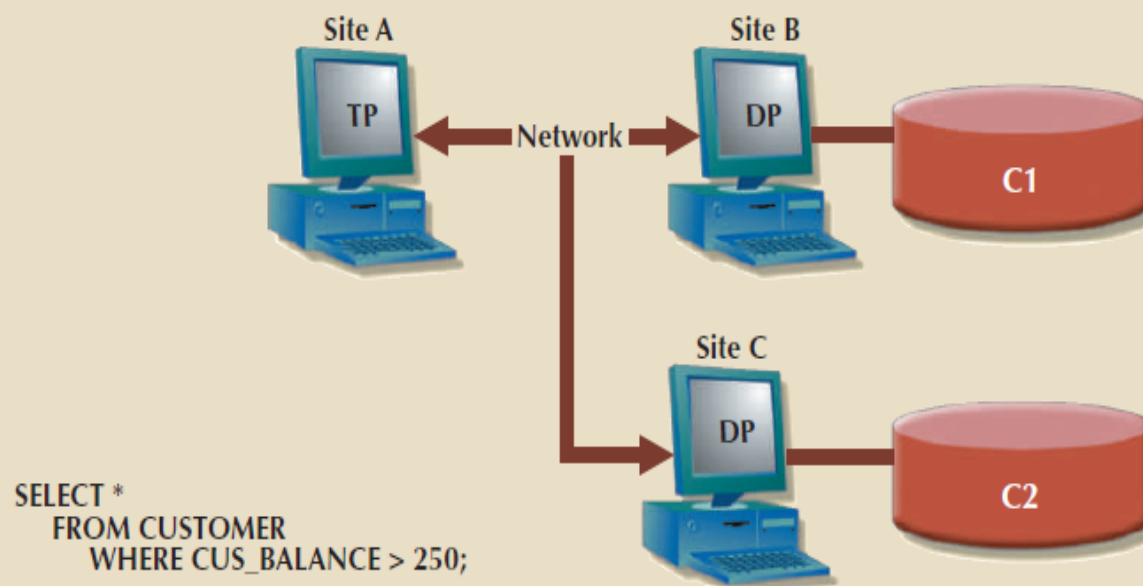
A distributed request

FIGURE 12.12 A DISTRIBUTED REQUEST



A distributed request

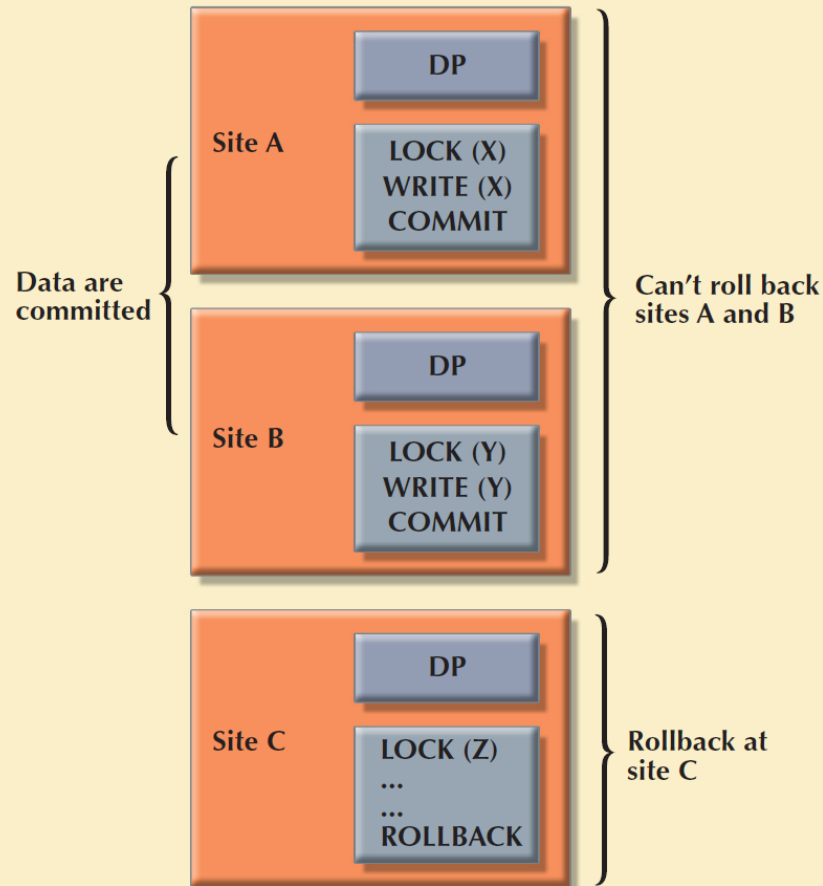
FIGURE 12.13 ANOTHER DISTRIBUTED REQUEST



Distributed Concurrency Control

- Concurrency control is important in distributed databases environment
- Due to multi-site multiple-process operations that create inconsistencies and deadlocked transactions
- A final COMMIT must not be issued until all sites have committed their parts of the transaction.

Figure 12.14 - The Effect of Premature COMMIT



Two-Phase Commit Protocol (2PC)

- Guarantees if a portion of a transaction operation cannot be committed, all changes made at the other sites will be undone
 - To maintain a consistent database state
- Requires that each DP's transaction log entry be written before database fragment is updated
- DO-UNDO-REDO protocol: Roll transactions back and forward with the help of the system's transaction log entries

DO-UNDO-REDO protocol

- The DO-UNDO-REDO protocol defines three types of operations:
- **DO** performs the operation and records the “before” and “after” values in the transaction log.
- **UNDO** reverses an operation, using the log entries written by the DO portion of the sequence.
- **REDO** redoes an operation, using the log entries written by the DO portion of the sequence.

Two-Phase Commit Protocol (2PC)

- **Write-ahead protocol:** Forces the log entry to be written to permanent storage before actual operation takes place
- Defines operations between coordinator and subordinates
- Phases of implementation
 - Preparation
 - The final COMMIT

Two-Phase Commit Protocol (2PC)

- Phase 1: Preparation
- The coordinator sends a PREPARE TO COMMIT message to all subordinates.
- 1. The subordinates receive the message, write the transaction log using the write-ahead protocol, and send an acknowledgment message (YES/PREPARED TO COMMIT or NO/NOT PREPARED) to the coordinator.
- 2. The coordinator makes sure that all nodes are ready to commit, or it aborts the action.
- If all nodes are PREPARED TO COMMIT, the transaction goes to Phase 2.
- If one or more nodes reply NO or NOT PREPARED, the coordinator broadcasts an ABORT message to all subordinates.

Two-Phase Commit Protocol (2PC)

- Phase 2: The Final COMMIT
- 1. The coordinator broadcasts a COMMIT message to all subordinates and waits for the replies.
- 2. Each subordinate receives the COMMIT message and then updates the database using the DO protocol.
- 3. The subordinates reply with a COMMITTED or NOT COMMITTED message to the coordinator.
- If one or more subordinates do not commit, the coordinator sends an ABORT message, thereby forcing them to UNDO all changes.

Performance and Failure Transparency

- Performance transparency: Allows a DDBMS to perform as if it were a centralized database
- Failure transparency: Ensures the system will operate in case of network failure
- Considerations for resolving requests in a distributed data environment
 - Data distribution
 - Data replication
 - Replica transparency: DDBMS's ability to hide multiple copies of data from the user

Performance and Failure Transparency

- Network and node availability
 - Network latency: **delay** imposed by the amount of time required for a data packet to make a round trip
 - Network partitioning: **delay** imposed when nodes become suddenly unavailable due to a network failure

Distributed Database Design

Data fragmentation

- How to partition database into fragments

Data replication

- Which fragments to replicate

Data allocation

- Where to locate those fragments and replicas

Data Fragmentation

- Breaks single object into many segments
 - Information is stored in distributed data catalog (DDC)
- Strategies
 - Horizontal fragmentation: Division of a relation into subsets (fragments) of tuples (rows)
 - Vertical fragmentation: Division of a relation into attribute (column) subsets
 - Mixed fragmentation: Combination of horizontal and vertical strategies

Data Replication

- Data copies stored at multiple sites served by a computer network
- Mutual consistency rule: Replicated data fragments should be identical
- Styles of replication
 - Push replication
 - Pull replication
- Helps restore lost data

Data Replication Scenarios

Fully replicated database

- Stores multiple copies of each database fragment at multiple sites

Partially replicated database

- Stores multiple copies of some database fragments at multiple sites

Unreplicated database

- Stores each database fragment at a single site

Data Allocation Strategies

Centralized data allocation

- Entire database stored at one site

Partitioned data allocation

- Database is divided into two or more disjointed fragments and stored at two or more sites

Replicated data allocation

- Copies of one or more database fragments are stored at several sites

The CAP Theorem

- CAP stands for:
 - Consistency
 - Availability
 - Partition tolerance
- Basically available, soft state, eventually consistent (BASE)
 - Data changes are not immediate but propagate slowly through the system until all replicas are consistent

Table 12.8 - Distributed Database Spectrum

TABLE 12.8

DISTRIBUTED DATABASE SPECTRUM

DBMS TYPE	CONSISTENCY	AVAILABILITY	PARTITION TOLERANCE	TRANSACTION MODEL	TRADE-OFF
Centralized DBMS	High	High	N/A	ACID	No distributed data processing
Relational DBMS	High	Relaxed	High	ACID (2PC)	Sacrifices availability to ensure consistency and isolation
NoSQL DDBMS	Relaxed	High	High	BASE	Sacrifices consistency to ensure availability
NewSQL DDBMS	High	High	Relaxed	ACID	Sacrifices partition tolerance to ensure transaction consistency and availability

C. J. Date's Twelve Commandments for Distributed Databases

- Local site independence
- Central site independence
- Failure independence
- Location transparency
- Fragmentation transparency
- Replication transparency

C. J. Date's Twelve Commandments for Distributed Databases

- Distributed query processing
- Distributed transaction processing
- Hardware independence
- Operating system independence
- Network independence
- Database independence